

★ WEEK 8 · PROBLEM SET

# Finance Applications: Three Hands-On Problems

**Fire a Neuron · Solve XOR · Walk Downhill**

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

---

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

# Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

## 1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

*one slide*

## 2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

*one slide*

## 3 · Hand Calc

pencil first

Work every number by hand in a table.  
Small  $n$ , clean arithmetic, no calculator fear.

*1–2 slides*

## 4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

*one slide*

**Hand first, machine second — you check the machine, never the other way around.**

# Where the Data Comes From

Public / in-script sources first — the prompts generate or fetch free data; CSVs are hand-calc references

| Problem               | Primary source (public · no signup)                 | Provided fallback file          |
|-----------------------|-----------------------------------------------------|---------------------------------|
| P1 · One neuron       | § 5 in-script set (make_classification, fixed seed) | — z values printed on the slide |
| P2 · XOR net          | truth table generated in-script (4 rows)            | fml_w8_p2_xor.csv (hand-calc)   |
| P3 · Gradient descent | $E(w)$ defined in-script — pure numpy               | — numbers printed on the slides |
| Real-data extension   | sklearn fetch_openml('creditcard') — free, no key   | —                               |

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

# P1

## PROBLEM 1 · ONE NEURON

# The Credit Score Was a Neuron All Along

$w = (-1, +2)$ ,  $b = 0$  — fire it with a step, then with a sigmoid

# Three Applicants Through One Neuron

Inputs (Income, Debt); weights  $w = (-1, +2)$ , bias  $b = 0$ . Fire with step, then with sigmoid

| Applicant | Income | Debt | Actually defaulted?       |
|-----------|--------|------|---------------------------|
| A         | 1.0    | 0.8  | Yes                       |
| B         | 2.0    | 0.5  | No                        |
| G         | 1.1    | 0.5  | Yes (Week 1's borderline) |

## Helper (e-values)

- $e^{(-0.6)} \approx 0.549 \cdot e^{(1.0)} \approx 2.718 \cdot e^{(0.1)} \approx 1.105$ .
- $\text{step}(z) = 1$  if  $z > 0$ , else 0.

## Your deliverables

- $z = w \cdot x + b$  for each applicant.
- $\text{step}(z)$  and  $\text{sigmoid}(z)$  outputs.
- One sentence: which Week 5 model did this neuron just become?

# The Tool: Weigh, Add, Fire

The dot product is the score; the activation is the personality

$$y = f(w \cdot x + b) \quad w = (-1, +2), b = 0 \Rightarrow z = 2 \cdot \text{Debt} - \text{Income} \quad (\text{Week 1's score IS a neuron})$$

## Reading the weights

- $w = (-1, +2)$ : income pushes risk DOWN, debt pushes it UP, twice as hard.
- $b = 0$ : no built-in lean either way.
- The neuron is a re-branding of  $z = 2 \cdot \text{Debt} - \text{Income}$  from Week 1.

## Lab guide — per applicant

- Step 1:  $z = (-1) \cdot \text{Income} + 2 \cdot \text{Debt}$ .
- Step 2:  $\text{step}(z)$  — fire or silent.
- Step 3:  $\text{sigmoid}(z) = 1/(1+e^{(-z)})$  using the helpers.

# Three Forward Passes

Same  $z$ , two activations — a hard opinion and a soft one

| Applicant | $z = 2 \cdot \text{Debt} - \text{Income}$ | $\text{step}(z)$ | $\text{sigmoid}(z)$ | Read                               |
|-----------|-------------------------------------------|------------------|---------------------|------------------------------------|
| A         | $1.6 - 1.0 = +0.6$                        | 1 — FIRE         | $1/1.549 = 0.65$    | risky, and 65% sure                |
| B         | $1.0 - 2.0 = -1.0$                        | 0 — silent       | $1/3.718 = 0.27$    | comfortably safe                   |
| G         | $1.0 - 1.1 = -0.1$                        | 0 — silent       | $1/2.105 = 0.47$    | a coin flip the step can't express |

step: A fires, B silent, G silent      sigmoid: 0.65, 0.27, 0.47 (= Week 5, exactly)

# One Neuron = Logistic Regression, Verified by Hand

The sigmoid outputs are Week 5's exact numbers — 0.65, 0.27, 0.47

**+0.6**

A's  $z$  — the only positive score; the step fires

**0.47**

G's sigmoid — the borderline the hard step throws away

**= W5**

Sigmoid neuron  $\equiv$  logistic regression, digit for digit

**$\times 10^6$**

Stack millions of these bricks — that is deep learning

## Step vs sigmoid

- The step gives a verdict; the sigmoid gives a confidence.
- Training needs gradients — another reason the smooth curve won.

## What one neuron cannot do

- It draws exactly one line through clue-space.
- P2 shows a rule no single line can express — and fixes it.

# Scale It: Watch a Neuron Learn Its Weights

Paste the metaprompt — Claude Code trains the neuron instead of hand-setting w

## [INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42),  
   standardizes, splits train/test;  
2) fits (a) LogisticRegression and (b) an  
   MLPClassifier with hidden_layer_sizes=() - a bare  
   single neuron - and shows their test F1 match;  
3) prints the learned weights of both side by side and  
   checks they agree in sign and rough size;  
4) then fits MLPClassifier((32, 16)) and reports the  
   F1 gain (if any) from depth on this tabular data;  
5) three-sentence verdict: what did depth buy here,  
   and why Week 1 predicts the gain is modest.
```

## [OUTPUT] What to expect

```
bare neuron F1 ~ logistic  
F1 (they are the same  
model)  
weights agree in sign  
deep MLP: small or no gain  
on this thin noisy table  
  
Your 3-pass table, learned  
from 20,000 rows.
```

# P2

## PROBLEM 2 · XOR

# The Rule One Line Cannot Draw

'Exactly one clue fires' — 1969's complaint, solved with two layers

# A Fraud Rule That Breaks a Single Neuron

Flag when EXACTLY ONE of (foreign  $x_1$ , midnight  $x_2$ ) is on. Both on = a traveler (normal); both off = normal

| $x_1$ foreign | $x_2$ midnight | Desired flag $y$             |
|---------------|----------------|------------------------------|
| 0             | 0              | 0 — normal                   |
| 1             | 0              | 1 — FLAG (lone odd signal)   |
| 0             | 1              | 1 — FLAG (lone odd signal)   |
| 1             | 1              | 0 — traveler pattern, normal |

## Your deliverables

- Show no single neuron works: adding the two 'flag' rows forces the (1,1) row to fire too.
- Forward-pass the two-layer net  $h_1 = \text{step}(x_1 + x_2 - 0.5)$ ,  $h_2 = \text{step}(x_1 + x_2 - 1.5)$ ,  $y = h_1 - h_2$  over all four rows.

# The Tool: Build OR and AND, Then Subtract

Layer 1 makes two simple opinions; layer 2 combines them into the rule

$$h_1 = \text{step}(x_1 + x_2 - 0.5) \quad h_2 = \text{step}(x_1 + x_2 - 1.5) \quad y = h_1 - h_2$$

## Why one line must fail

- Firing on (1,0) and (0,1) needs  $w_1 > t$  and  $w_2 > t$ .
- Add them:  $w_1 + w_2 > 2t \geq t$  (for any sensible  $t \geq 0$ ).
- So (1,1) fires too — the traveler gets flagged. Contradiction.

## Lab guide — per row

- Step 1:  $h_1$  = OR of the clues (threshold 0.5).
- Step 2:  $h_2$  = AND of the clues (threshold 1.5).
- Step 3:  $y = h_1 - h_2$  — 'at least one, but not both'.

# Four Rows Through Two Layers

OR minus AND = exactly-one — the bend a single line cannot make

| $(x_1, x_2)$ | $x_1 + x_2$ | $h_1 = \text{step}(\cdot - 0.5)$ 'OR' | $h_2 = \text{step}(\cdot - 1.5)$ 'AND' | $y = h_1 - h_2$      |
|--------------|-------------|---------------------------------------|----------------------------------------|----------------------|
| (0, 0)       | 0           | 0                                     | 0                                      | 0 — normal           |
| (1, 0)       | 1           | 1                                     | 0                                      | 1 — FLAG             |
| (0, 1)       | 1           | 1                                     | 0                                      | 1 — FLAG             |
| (1, 1)       | 2           | 1                                     | 1                                      | 0 — traveler, normal |

$(0, 0) \rightarrow 0$      $(1, 0) \rightarrow 1$      $(0, 1) \rightarrow 1$      $(1, 1) \rightarrow 0$     (XOR: exactly one clue fires)

# Depth Bought a Bend — and Ended an AI Winter

The 1969 critique was right about one neuron, and wrong about the idea

## 0 lines

Single-neuron solutions to XOR — proven impossible by adding two rows

## 2 layers

OR and AND, then subtract — four rows, solved

## 1 bend

What depth fundamentally adds — stack bends, get any shape

## 1986

Backprop makes such layers LEARNABLE, not hand-set

### The finance reading

- Real fraud rules are full of 'unless together' exceptions.
- Interaction effects are exactly what depth (and W7's trees) capture.

### We cheated, honestly

- We SET the weights; real nets learn them by backprop (P3's walk).
- The point survives: expressiveness comes from stacking.

# Scale It: Let a Net LEARN XOR

Paste the metaprompt — Claude Code trains what we hand-built and draws the bend

## [INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) creates the XOR dataset: 2000 points sampled near the four corners (0,0),(1,0),(0,1),(1,1) with Gaussian noise 0.1, labels = XOR of the corners;
- 2) fits LogisticRegression (one neuron) and prints its accuracy - show it is stuck near 50-75%;
- 3) fits MLPClassifier(hidden\_layer\_sizes=(4,), activation='relu', max\_iter=2000) and prints its accuracy - near 100%;
- 4) plots the decision boundary of both models side by side over the point cloud;
- 5) three-sentence verdict: what the hidden layer's bend does that the single line cannot, in this week's language.

## [OUTPUT] What to expect

logistic: one straight line,  
~50-75% accuracy  
small MLP: a bent boundary  
hugging two corners, ~100%  
the picture IS the lesson:  
depth = bends

Your 4-row table, learned  
from 2,000 noisy points.

# P3

## PROBLEM 3 · GRADIENT DESCENT

# Three Step Sizes, Three Fates

$E(w) = (w - 3)^2$  from  $w = 0$  — steady, perfect, and divergent

# Walk Down One Parabola, Three Ways

Error  $E(w) = (w - 3)^2$ , start at  $w = 0$ . Take 3 steps at  $\eta = 0.25$ , then  $\eta = 0.5$ , then  $\eta = 1.1$

**The gradient is  $dE/dw = 2(w - 3) = -2 \times \text{gap}$ . Each update is  $w \leftarrow w + 2\eta \cdot (3 - w)$  — so the gap to the valley scales by  $(1 - 2\eta)$ .**

## Given

- Valley (the answer):  $w^* = 3$ .
- Start:  $w_0 = 0$ , so the initial gap is 3.
- Three learning rates to try: 0.25, 0.5, 1.1.

## Your deliverables

- The  $w$  path for three steps at each  $\eta$ .
- The gap after each step and its scaling factor  $(1 - 2\eta)$ .
- One sentence per  $\eta$ : steady walker, lucky jumper, or diverger?

# The Tool: One Update Rule, One Scaling Factor

On a parabola the whole drama collapses to how  $(1 - 2\eta)$  treats the gap

$$E(w) = (w - 3)^2 \quad w \leftarrow w - \eta \frac{dE}{dw} = w + 2\eta(3 - w) \quad \text{gap} \leftarrow (1 - 2\eta) \cdot \text{gap}$$

## Reading the factor

- $|1 - 2\eta| < 1$ : the gap shrinks — convergence.
- $1 - 2\eta = 0$  ( $\eta = 0.5$ ): the gap dies in ONE step — parabola luck.
- $|1 - 2\eta| > 1$  ( $\eta > 1$ ): the gap GROWS — each step overshoots harder.

## Lab guide — per $\eta$

- Step 1:  $\text{gap} = 3 - w$ .
- Step 2:  $w \leftarrow w + 2\eta \cdot \text{gap}$ ; write the new  $w$  and new gap.
- Step 3: repeat three times; note the pattern.

# Nine Updates, Three Stories

Steady halving, a one-step bullseye, and a bounce that never lands

| $\eta$ | Factor $1 - 2\eta$ | w path from 0                                          | Gap path                                                | Story                    |
|--------|--------------------|--------------------------------------------------------|---------------------------------------------------------|--------------------------|
| 0.25   | 0.5                | $0 \rightarrow 1.5 \rightarrow 2.25 \rightarrow 2.625$ | $3 \rightarrow 1.5 \rightarrow 0.75 \rightarrow 0.375$  | steady — halves forever  |
| 0.5    | 0                  | $0 \rightarrow 3.0 \rightarrow 3.0 \rightarrow 3.0$    | $3 \rightarrow 0$                                       | bullseye in one step     |
| 1.1    | -1.2               | $0 \rightarrow 6.6 \rightarrow -1.32 \rightarrow 8.18$ | $3 \rightarrow -3.6 \rightarrow 4.32 \rightarrow -5.18$ | overshoot, flip, DIVERGE |

$\eta = 0.25$ : gap halves       $\eta = 0.5$ : one perfect step       $\eta = 1.1$ : gap  $\times 1.2$  (diverges)

# The Learning Rate IS the Model's Temperament

One factor,  $|1 - 2\eta|$ , decided steady, perfect, or explosive

## 0.5

$\eta = 0.25$ 's gap factor —  
reliable geometric  
progress

## 1 step

$\eta = 0.5$ 's bullseye —  
real losses are never  
this kind

## $\times 1.2$

$\eta = 1.1$ 's gap growth —  
the bouncing  
divergence of Q3

## $= W7$ 's $\eta$

Boosting's careful  
half-steps were this  
same idea

## Why small $\eta$ wins in practice

- Real loss surfaces are bumpy, not one clean parabola.
- Careful steps average over the bumps instead of amplifying them.

## Backprop's role

- In deep nets the gradient per weight comes from backprop.
- Then every weight takes exactly this walk, simultaneously.

# Scale It: See All Three Fates on One Chart

Paste the metaprompt — Claude Code animates the walk and then trains a real net

## [INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:
1) implements gradient descent on  $E(w) = (w - 3)^2$ 
   from  $w = 0$  for eta in [0.25, 0.5, 0.9, 1.1], 15
   steps each, in pure numpy;
2) plots the parabola with each walk's w path overlaid
   (markers joined by lines) - one panel per eta;
3) prints a table: eta | gap factor 1-2*eta | gap
   after 15 steps;
4) then trains MLPClassifier on the class dataset
   (make_classification, 20000 x 20, weights
   [0.99, 0.01], random_state=42) with
   learning_rate_init in [0.0001, 0.001, 0.1] and
   plots the three loss_curve_ traces;
5) three-sentence verdict linking the toy factor to
   the real loss curves.
```

## [OUTPUT] What to expect

```
0.25: smooth slide to 3
0.5: instant landing
0.9: converging zigzag
1.1: exploding zigzag

real net: 0.1 bounces or
         plateaus, 0.001 descends

Your 9-update table, drawn
and then made real.
```

# Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

**1**

**One neuron = Week 5's logistic, digit for digit**

$z = +0.6/-1.0/-0.1 \rightarrow \text{sigmoid } 0.65/0.27/0.47$  — the credit score was a neuron all along.

**2**

**Depth buys bends: XOR in four rows**

OR minus AND flags 'exactly one clue' — the rule no single line can draw, and 1969's complaint answered.

**3**

**$\eta$  is temperament:  $0.5 \cdot \text{gap}$ , bullseye, or  $\times 1.2$  blow-up**

The factor  $(1 - 2\eta)$  told the whole story — and Week 7's boosting  $\eta$  was the same idea in disguise.

**Hand calculation first — then let Claude Code scale the same idea to real data.**